

Google Antigravity Customization Architecture

Global Roots (~/.gemini/config) vs. Workspace Roots (.agents/)

Executive Summary: Antigravity implements a dual-tier configuration hierarchy separating **Global Customizations** (machine-local, user-specific tools & configs residing in ~/.gemini/config/) from **Workspace Customizations** (project-specific, version-controlled guidelines & skills residing in .agents/). Workspace definitions take strict precedence over global definitions to ensure reproducible, team-aligned agent behavior.

1. Direct Side-by-Side Comparison

Attribute	Global Root (~/.gemini/config)	Workspace Root (.agents/)
Standard Location	/home/user/.gemini/config/ (Linux/Mac) C:\Users<user>\.gemini\config\ (Win)	<project-root>/.agents/ (Also accepts .agent/, _agents/)
Effective Scope	Machine-wide: Applies to <i>all</i> projects and conversations on the developer machine.	Project-scoped: Applies <i>only</i> when operating within this repository workspace.
VCS / Git Status	Uncommitted / Local: Stored in user home, private to machine.	Committed (VCS): Tracked in Git and shared across the entire team.
Loading Priority	Lower Priority: Serves as the developer's default fallback.	Highest Priority: Overrides global and built-in configurations.
Ideal Use Cases	Personal dev preferences, private API keys, system utilities, personal MCP servers.	Team coding standards, project build runbooks, CI/CD skills, project MCP configs.

2. Deep Dive: Global vs. Workspace Roots

A. Global Customization Root (~/.gemini/config/)

The Global Root provides environment-wide capabilities that a developer wants available across all coding sessions:

- **Personal Workflow Skills:** General automation runbooks (e.g. personal dotfile management, cross-repo git cherry-picking).
- **Private MCP Servers:** Connection to personal tools (e.g., local Obsidian notes, private password manager, personal cloud credentials).
- **Global Plugins & Aliases:** Shared plugin bundles installed globally on the developer machine.

B. Workspace Customization Root (.agents/ & Project Rules)

The Workspace Root encapsulates repository-specific intelligence, turning the agent into a domain expert on that specific codebase:

- **Team Coding Guidelines (Rules):** Enforced via GEMINI.md, AGENTS.md, or .agents/rules/*.md. Supports directory-level hierarchy.
- **Project Runbooks (Skills):** Repository deployment workflows, database migrations, custom test runner executions defined in .agents/skills/<name>/SKILL.md.
- **Project-Scoped MCP & Hooks:** mcp_config.json and hooks.json tailored specifically to project microservices or Docker stacks.

3. Precedence Hierarchy & Conflict Resolution

When Antigravity discovers multiple configurations or identically named skills/rules, it applies a deterministic 5-level precedence order (Highest to Lowest):

Priority	Source Level	Resolution Behavior & Precedence
----------	--------------	----------------------------------

1 (Highest)	Workspace Project Discovery	.agents/ and hierarchical GEMINI.md / AGENTS.md walking from CWD up to repo root.
2	Workspace Declared Configs	Explicitly declared entries in workspace skills.json or plugins.json.
3	Global Discovery	Files discovered within ~/.gemini/config/ (skills, rules, MCPs).
4	Built-in Customizations	Default skills and system tools packaged directly with Antigravity.
5 (Lowest)	Global Declared Configs	Explicit entries in global skills.json or plugins.json.

4. Directory Layouts & Progressive Context Loading

Antigravity enforces **Progressive Disclosure**: skills and model-decision rules are not dumped into the prompt window. Only lightweight metadata headers are initially indexed; full content is retrieved dynamically on demand.

Workspace Root (Repo / Project)	Global Root (~/.gemini/config/)
■■■■ .agents/	■■■■ skills/
■ ■■■■ skills/deploy/SKILL.md	■ ■■■■ global-cli/SKILL.md
■ ■■■■ rules/frontend-style.md	■■■■ plugins/
■ ■■■■ hooks.json	■■■■ mcp_config.json
■ ■■■■ mcp_config.json	■■■■ plugins.json
■■■■ GEMINI.md (or AGENTS.md)	
■■■■ src/ ...	

5. Best Practice Recommendations

- **Check in .agents/ to Git:** Treat workspace skills and rules as code artifacts so every team member benefits from identical agent capabilities.
- **Keep Sensitive Keys in Global:** Never commit private access tokens or machine-specific MCP credentials into workspace files; place them in ~/.gemini/config/.
- **Leverage Hierarchical Rules:** Place subsystem-specific rules near their respective code directories (e.g. src/frontend/GEMINI.md) to avoid context pollution.